

Object Oriented Programming - An Engineering Perspective

Sameh Muhammed



[LinkedIn](#)



[Youtube](#)

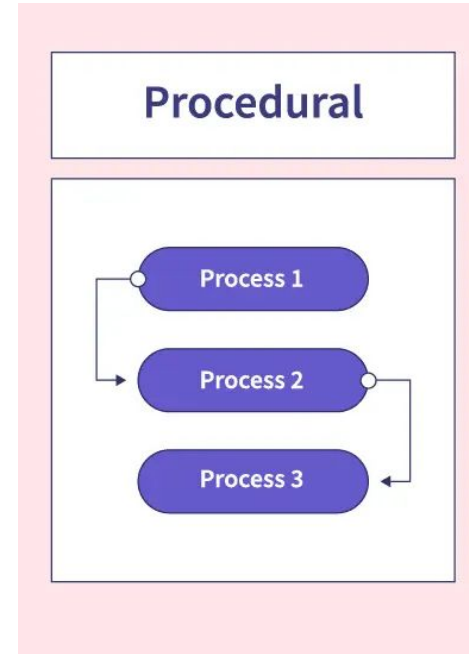


[adplist](#)

Course Outline

Before OOP : Structured Programming

Procedural programming is a programming paradigm built around the idea that programs are sequences of instructions to be executed. They focus heavily on splitting up programs into named sets of instructions called **procedures**, analogous to **functions**.

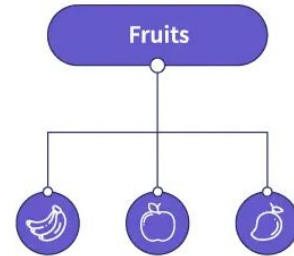


Hotel
Management
System
Demo

What is OOP?

Object-oriented programming (OOP) is a computer programming model that organizes software design around **data**, or **objects**, rather than **functions** and **logic**. An object can be defined as a data field that has unique attributes and behavior.

Object Oriented



OOP Principles

Encapsulation

Inheritance

Abstraction

Polymorphism

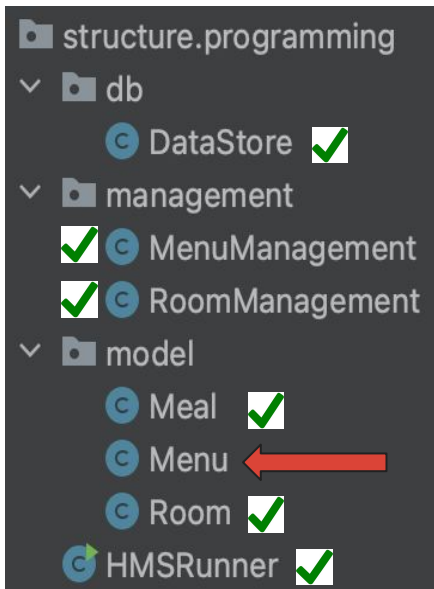
Encapsulation

Is about grouping related properties & behaviours into one **entity** and **hiding** unnecessary information.

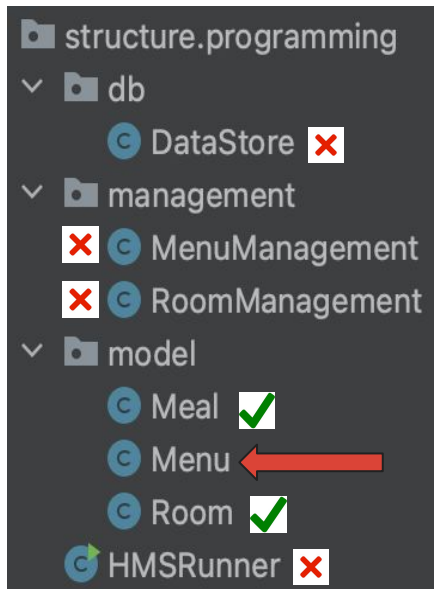


Data Hiding / Access Modifiers

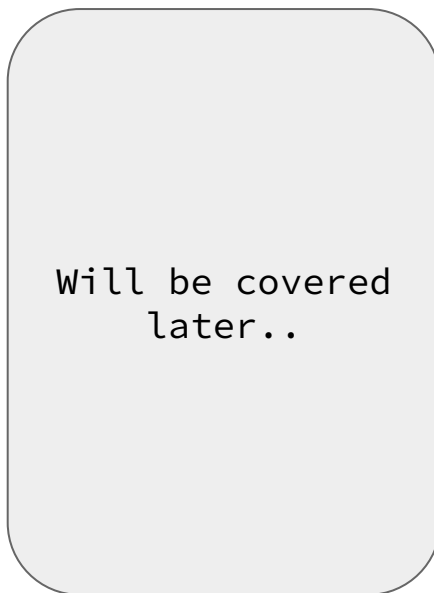
Public



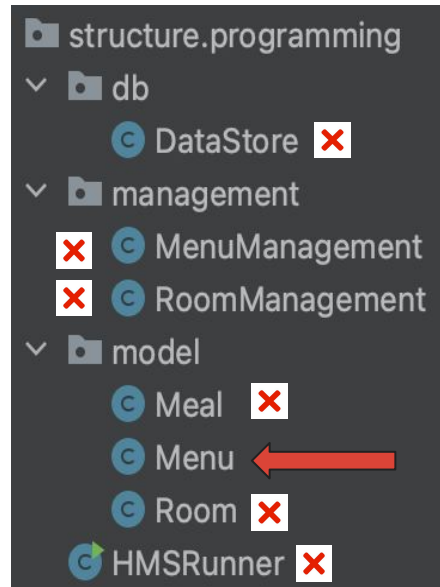
Package



Protected



Private



Encapsulation Guidelines

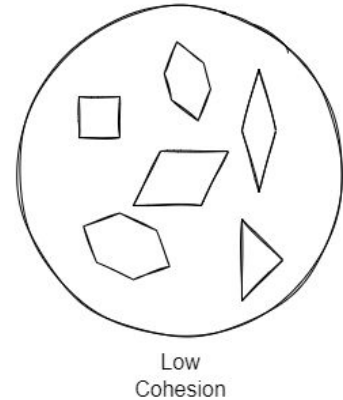
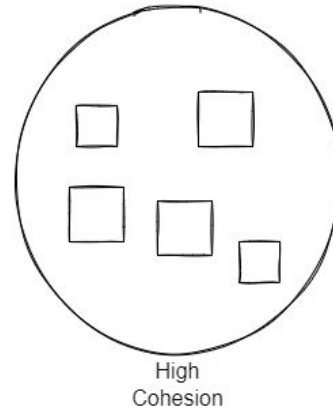
Cohesion

Single
Responsibility

Well Formed
Constructors

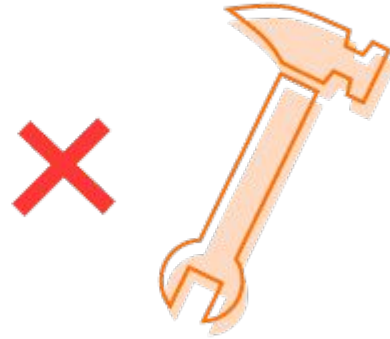
Cohesion

refers to the degree to which the elements inside a module belong together. In one sense, it is a measure of the **strength** of relationship between the methods and data of a class and some unifying purpose or concept served by that class.



Single Responsibility

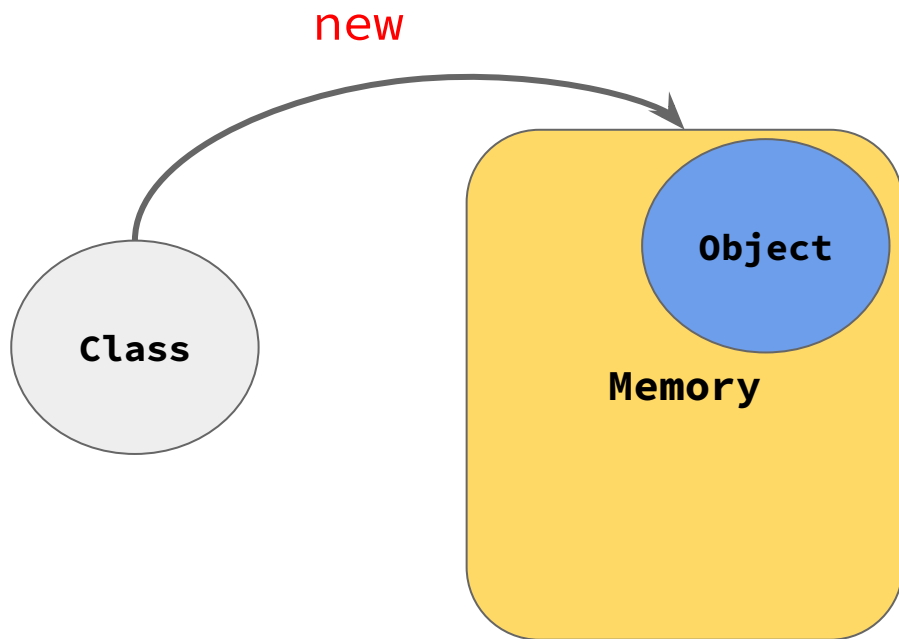
A class or function or modules should have only **one reason to change**. A class or function or modules should have only **one responsibility**.



Constructos

A constructor in Java is a **special method** that is used to initialize objects. The constructor is called when an object of a class is created.

```
ClassName objectName = new ClassName();
```



Initialization Safety

```


// class definition
class Room {

    Room() {
        // default constructor
    }
}

// class usage
Room room = new Room();
room.number = 1;
room.price = 250.0;
room.numberOfBeds = 2;

```

```


// class definition
class Room {

    //parameterized constructor
    Room(int number, double price, int numberOfBeds) {
        this.number = number;
        this.price = price;
        this.numberOfBeds = numberOfBeds;
    }

}

// class usage
Room room = new Room(1,250.0, 2);

```

Multiple Constructors

Is about providing multiple ways for object creation, each way is representing **special purpose**.

```
Room(int number, double price, int numberOfBeds, String amenities)
{
    this.number = number;
    this.price = price;
    this.numberOfBeds = numberOfBeds;
    this.amenities = amenities;
}
```

```
Room(int number, double price, int numberOfBeds) {
    this.number = number;
    this.price = price;
    this.numberOfBeds = numberOfBeds;
}
```

```
Room(int number) {
    this.number = number;
}
```

Factory Methods

Revealing constructor
intention/purpose through
descriptive name.

```
public static Room createRoomWithAmenities(int number, double price, int numberOfBeds,
String amenities) {

    Room room = new Room();
    room.setNumber(number);
    room.setPrice(price);
    room.setNumberOfBeds(numberOfBeds);
    room.setAmenities(amenities);

    return room;
}
```

```
public static Room createDefaultRoom(int number, double price, int numberOfBeds) {

    Room room = new Room();
    room.setNumber(number);
    room.setPrice(price);
    room.setNumberOfBeds(numberOfBeds);

    return room;
}
```

```
public static Room createEmptyRoom(int number) {

    Room room = new Room();
    room.setNumber(number);

    return room;
}
```

Factory Design Pattern & Builder Design Pattern

Factory Design Pattern

Is about creating a special class which encapsulate the creation logic of **group of classes**

Builder Design Pattern

Is about providing a way to create objects from a **specific class** from any combination of properties

Method Overload

allows different methods to have the same name, but different **signatures** where the signature can differ by the number of input parameters or type of input parameters, or a mixture of both.

Overloading

```
class Dog{  
    public void bark(){  
        System.out.println("woof ");  
    }  
  
    //overloading method  
    public void bark(int num){  
        for(int i=0; i<num; i++)  
            System.out.println("woof ");  
    }  
}
```

Same Method Name,
Different Parameter

Singleton Design Pattern

As we know that the constructor is kind of method, so what if we can make this method `private`?



Singleton Design Pattern

```
class Hotel {  
    private String name;  
    private Hotel(String name) {  
        this.name = name;  
    }  
  
    private static Hotel hotelInstance;  
    public static Hotel getHotel() {  
        if(hotelInstance == null) {  
            hotelInstance = new Hotel("OOPIn");  
        }  
        return hotelInstance;  
    }  
}
```

The diagram illustrates the Singleton Design Pattern implementation. It features a code editor window with Java code. Two red rounded rectangles highlight specific parts of the code: the first rectangle encloses the private constructor and the private instance variable, while the second rectangle encloses the public static method that checks for the existence of the instance and creates it if necessary. Two yellow ovals on the right side of the image provide labels: the top oval points to the first red rectangle and is labeled 'Normal class with private constructor', and the bottom oval points to the second red rectangle and is labeled 'Singleton design pattern implementation'. Blue arrows connect these ovals to their respective code sections.

**Normal class
with private
constructor**

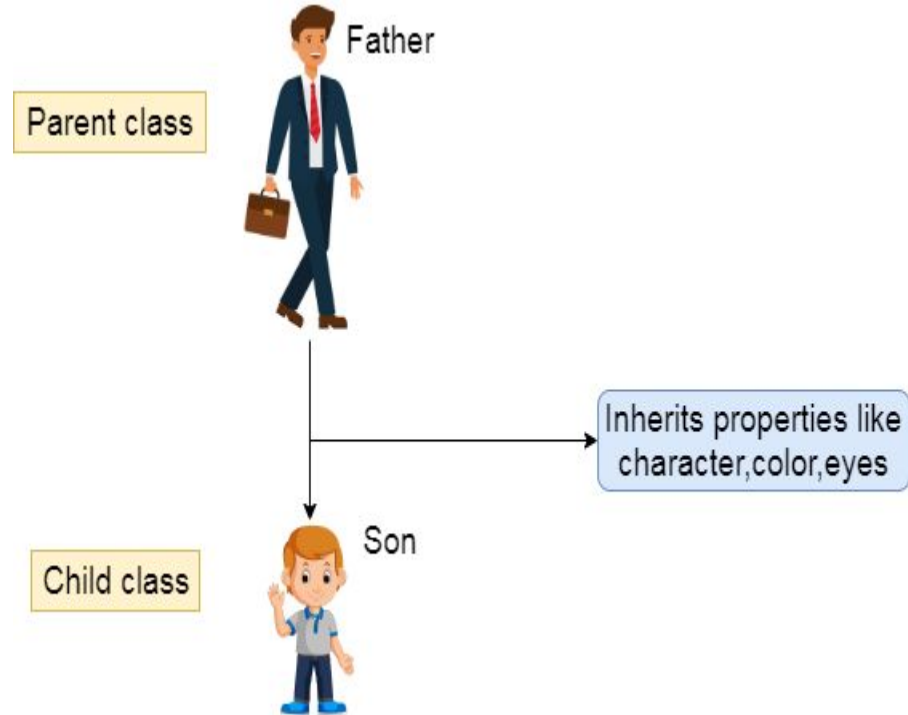
**Singleton
design pattern
implementation**

Inheritance

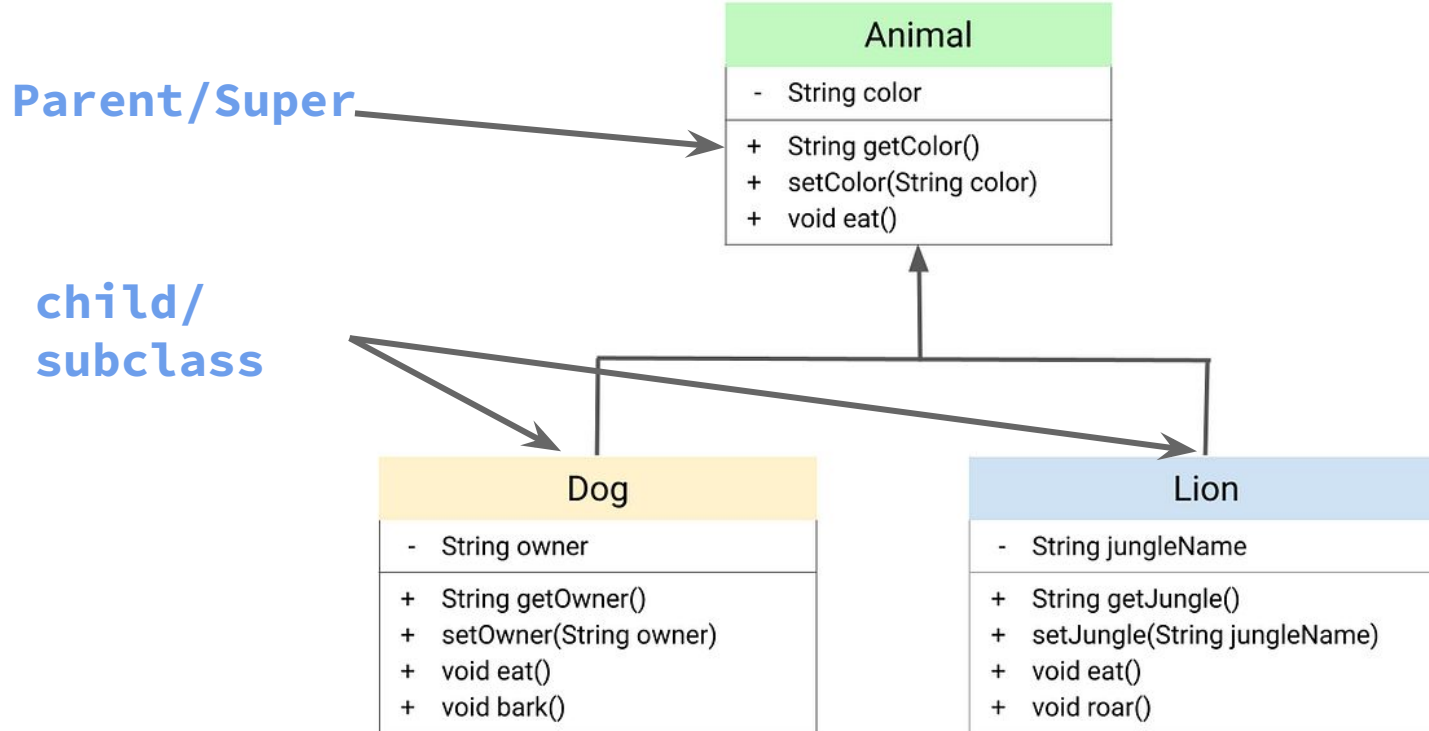
It is a mechanism where you are able to derive a class from another class for a hierarchy of classes that **share** a set of attributes and methods.

It describes “**Is A**” relationship between classes.

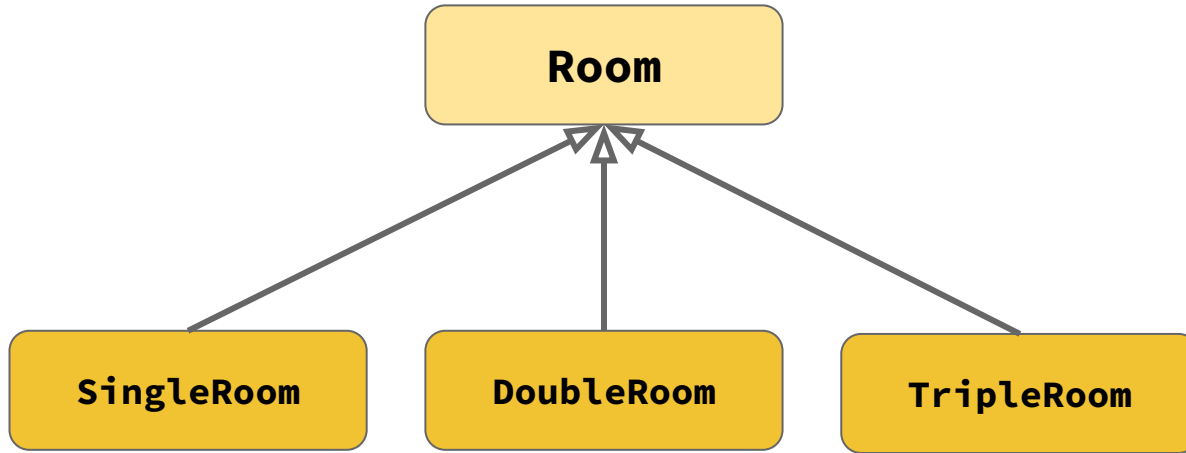
The main advantage of inheritance is **reusing** code.



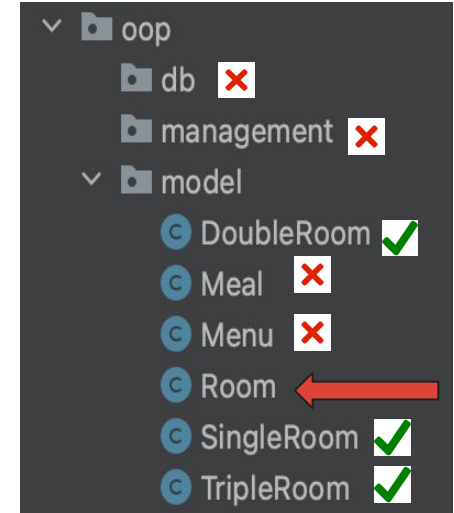
Inheritance Example



Protected Access Modifier

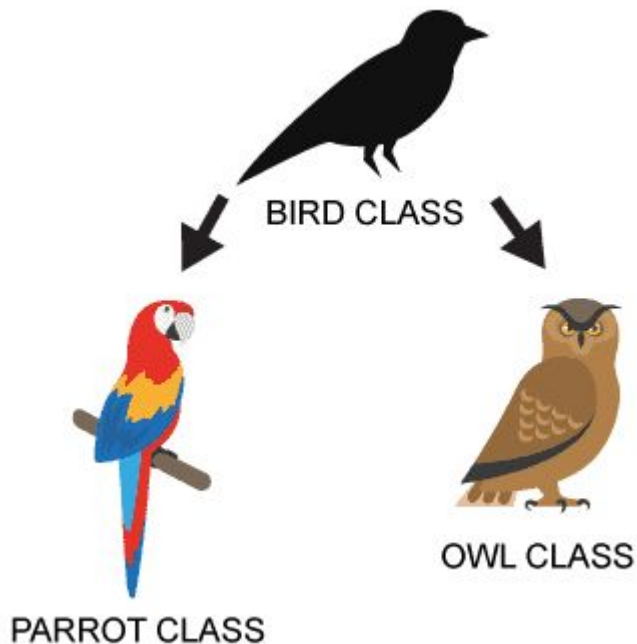


Protected

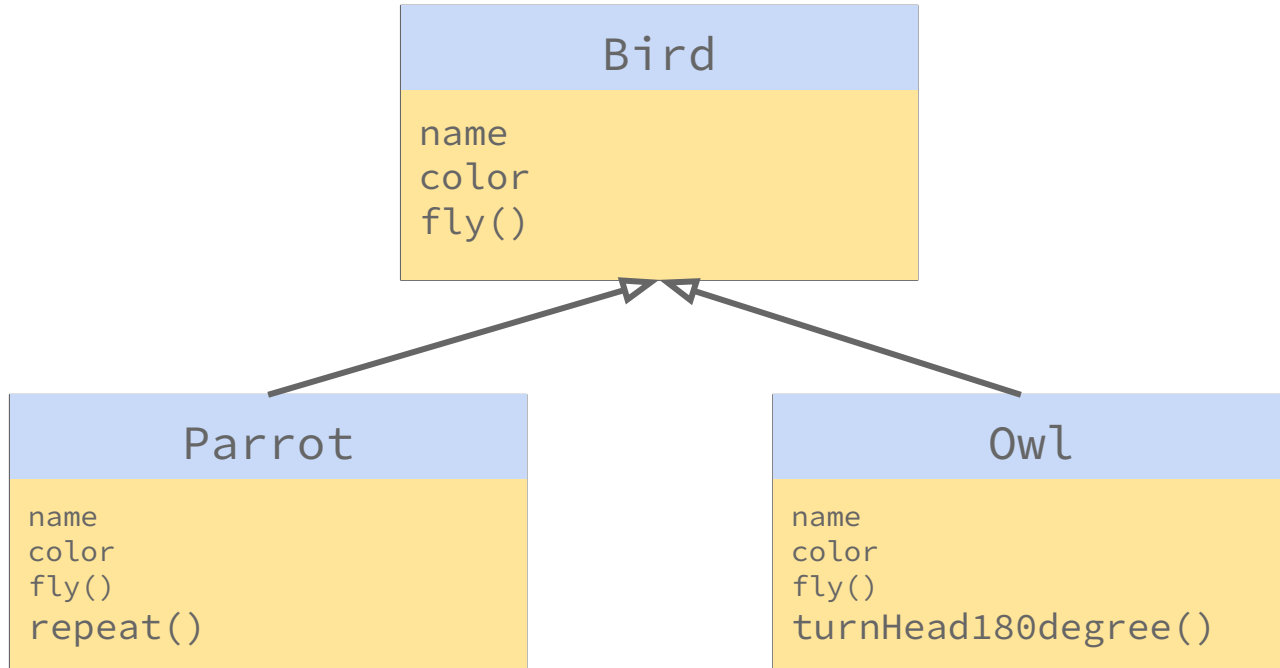


Can
Inheritance
be harmful?

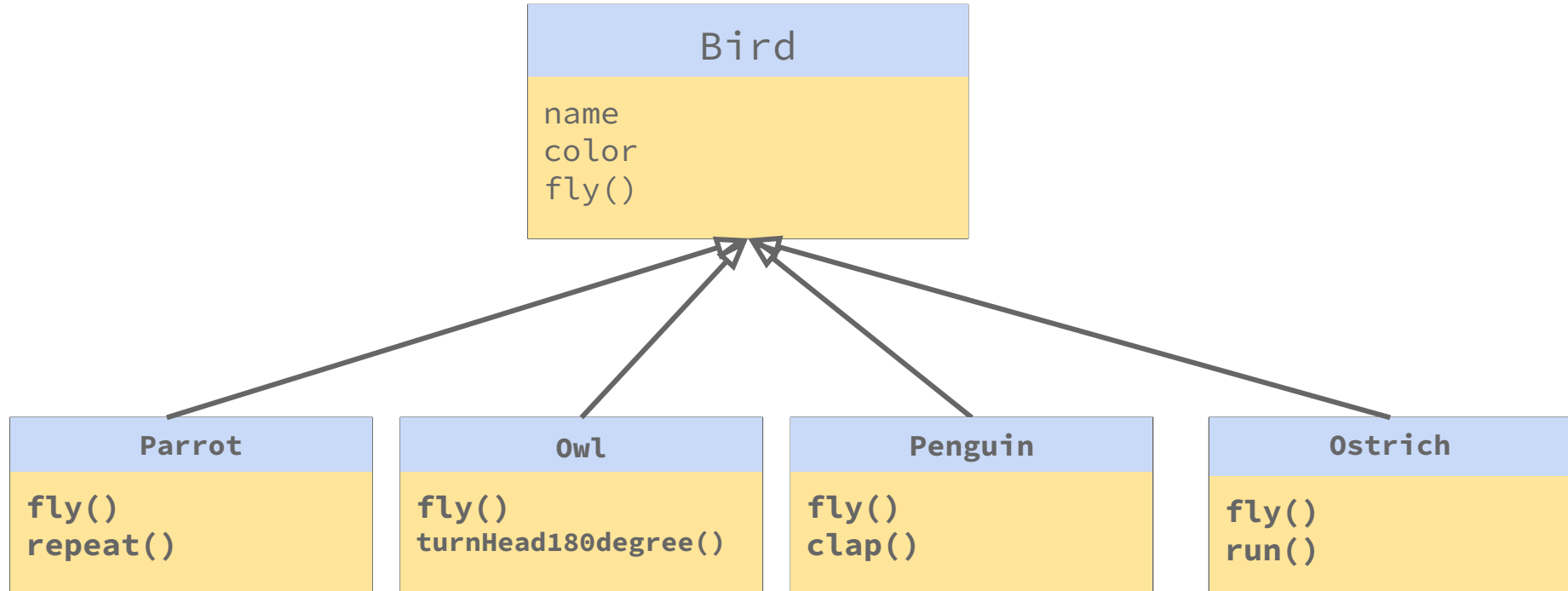
Let's Take This xample



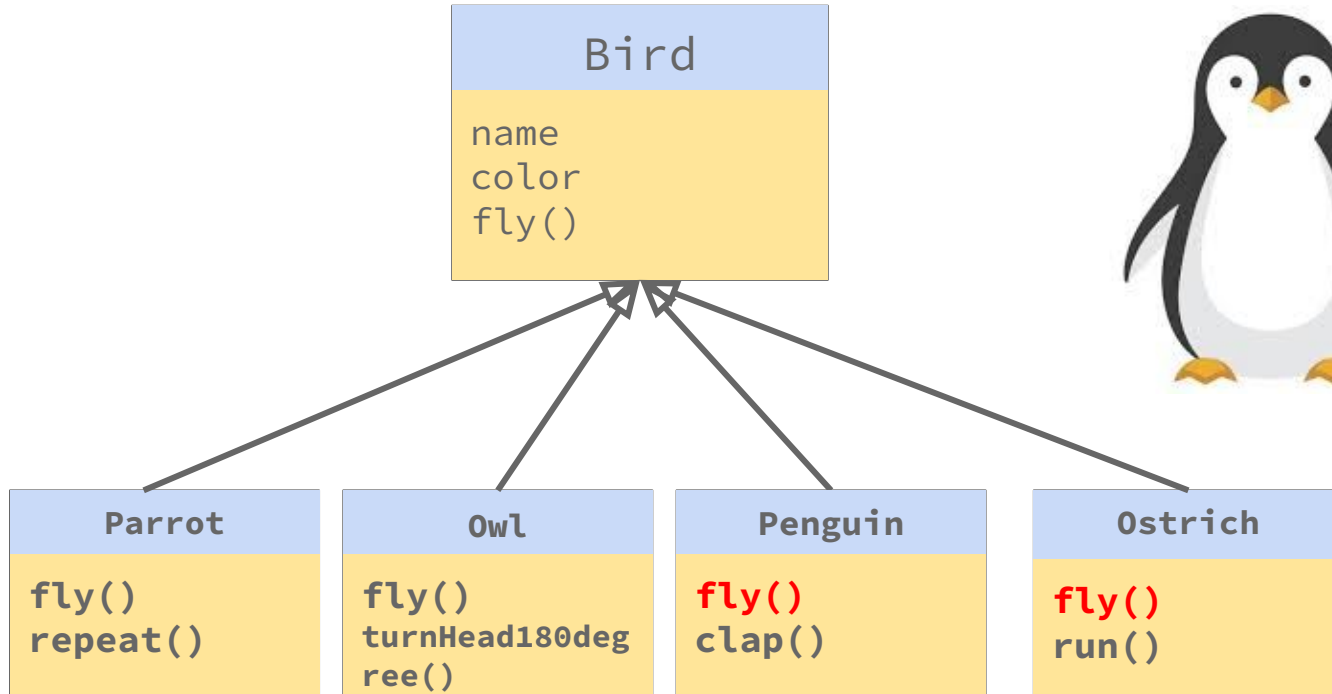
Class Diagram



Class Diagram Extended



Problem Detected



Let's think
about
solutions for
this problem

Proposed Solutions: Naive

- Remove fly method from parent class, and add it just in required classes.

Proposed Solutions: Naive

- Remove fly method from parent class, and add it just in required classes.
 - Will cause a lot of code duplication which can be tedious if there is an error or modification need to be done on that logic.
 - Will be tedious while adding more types of birds

Proposed Solutions: Naive

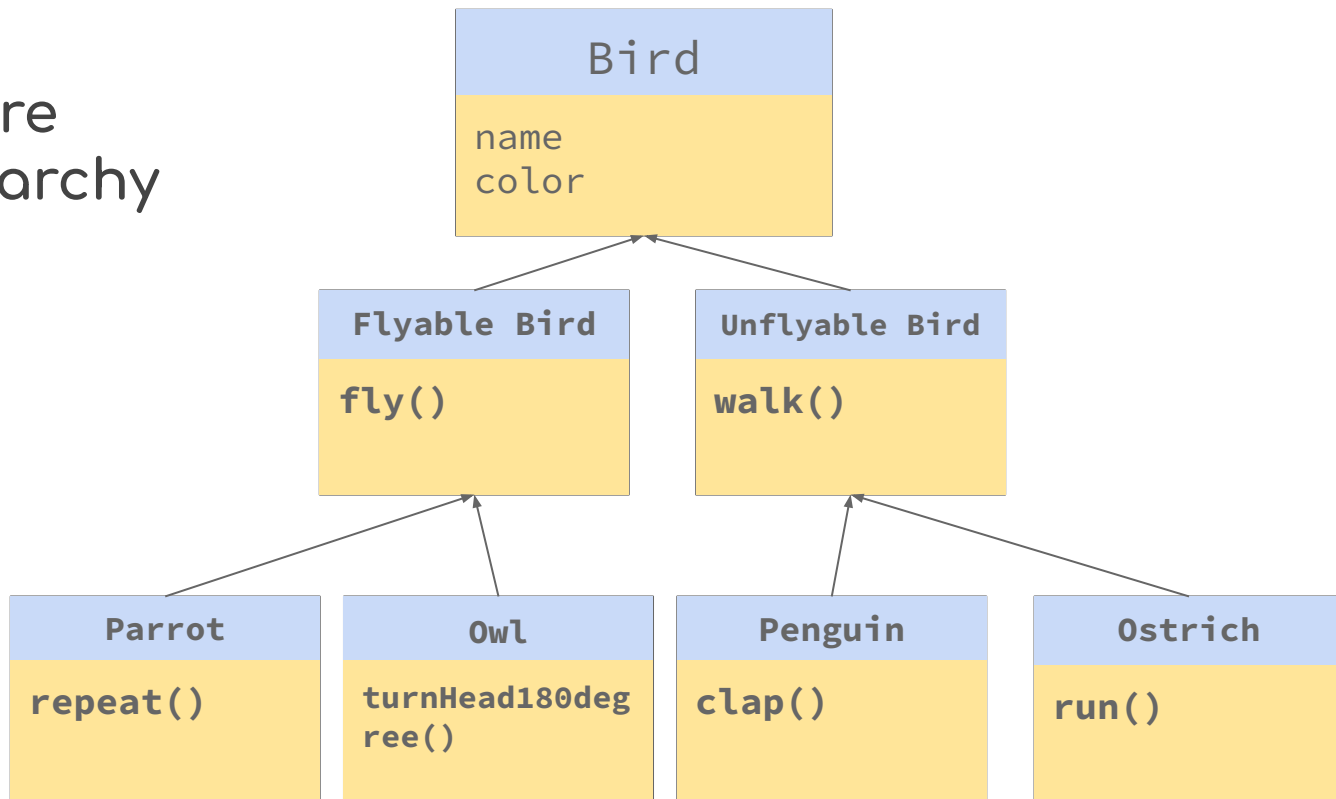
- Rewrite the logic of fly method in Ostrich & Penguin classes.

Proposed Solutions: Naive

- Rewrite the logic of fly method in Ostrich & Penguin classes.
 - In case of adding more types, you may forget to rewrite the logic in new classes and this will be silent problem you can't discover it till runtime as in compile time it will be correct

Proposed Solutions: Bit Advanced

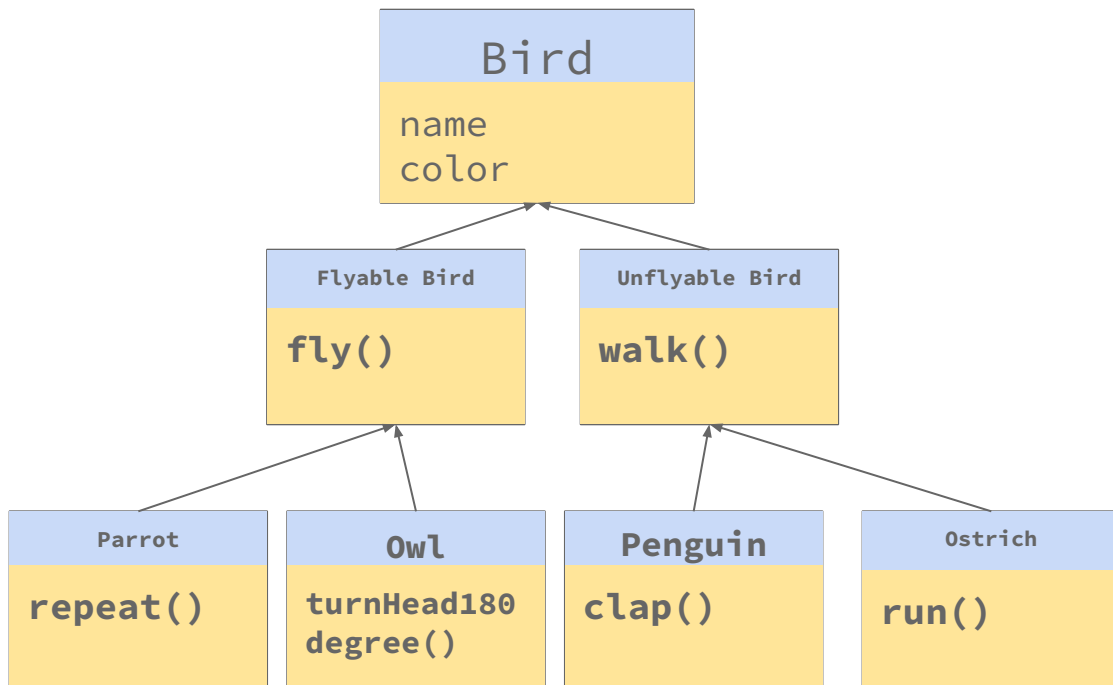
- Restructure class hierarchy



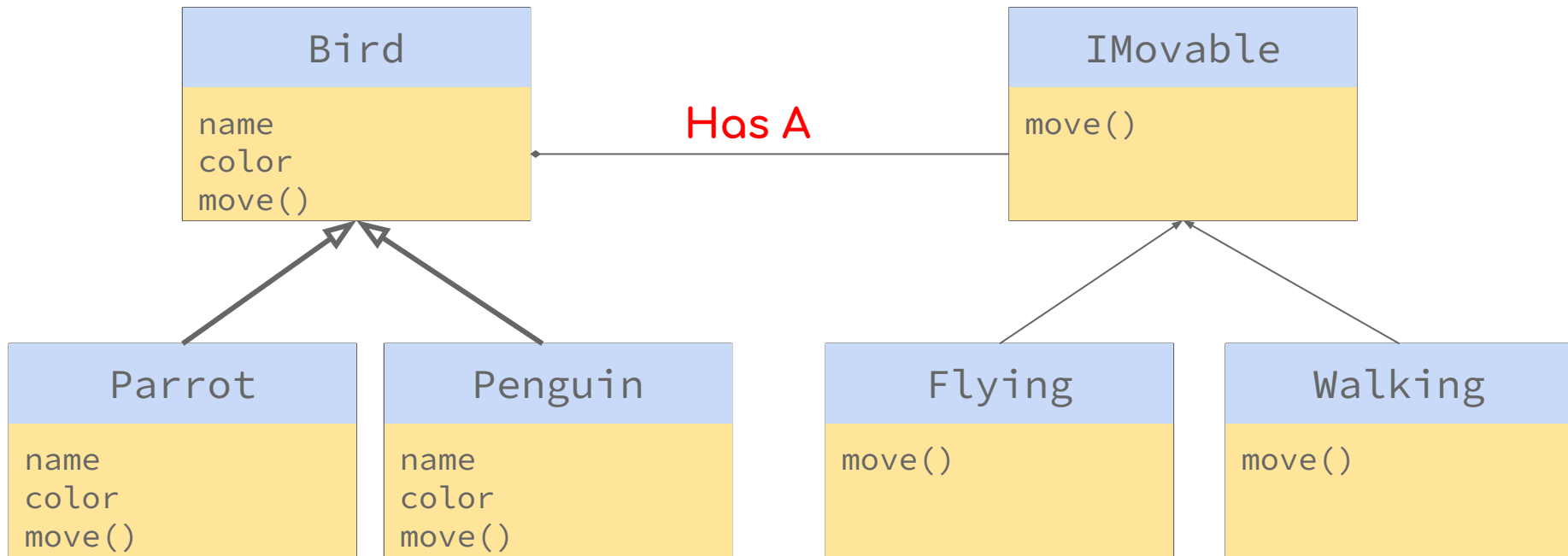
Proposed Solutions: Bit Advanced

- Restructure class hierarchy

- Will complicate class hierarchy as you now group classes based on specific behaviour



The solution : Use Composition



Code Example

Solution Notes

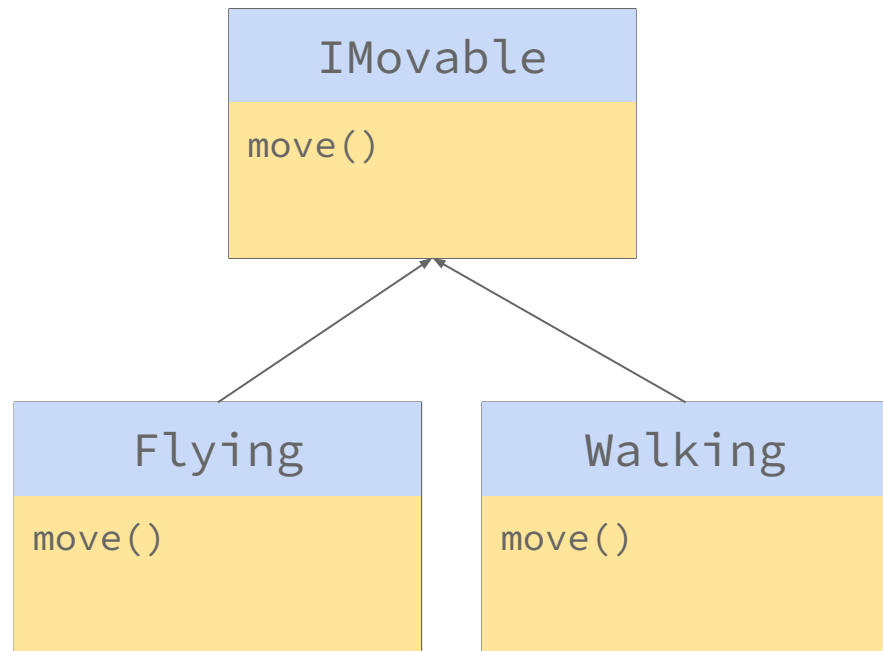
1- We used composition which represents a Has A relationship between 2 classes, which means that the class A consists of class B

2- Favour composition over inheritance which is a general rule good to follow as maintaining inheritance relationships can be hard most of the time.

3- The solution we applied is a real implementation of Open Closed principle in "SOLID" principles, which declare that your component should be open for extension but closed for modification.

Solution Notes: Strategy Design Pattern

The solution we applied is an implementation of **Strategy** Design Pattern, which can be defined as a behavioral design pattern that lets you define a **family of algorithms**, put each of them into a separate class, and make their objects interchangeable at runtime.



Abstraction

Abstraction is the process of **hiding** the **internal details** of an component from the outer world, and it has 2 types

- Data abstraction.
- Behaviour complexity abstraction.



Abstraction in Action

Access
Modifiers

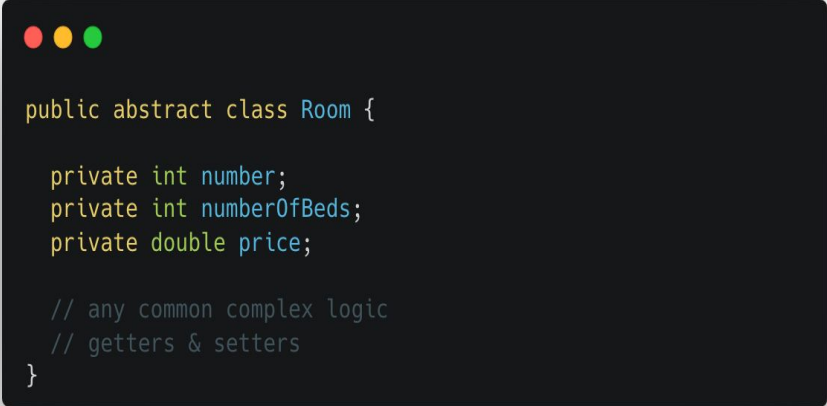
Abstract Class

Interface

Abstract
Method

Abstract Class

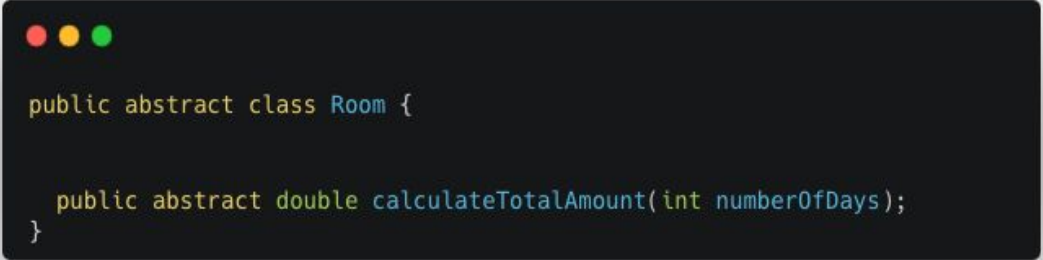
is a restricted class that **cannot be used** to create objects (to access it, it must be inherited from another class)



```
public abstract class Room {  
  
    private int number;  
    private int numberOfBeds;  
    private double price;  
  
    // any common complex logic  
    // getters & setters  
}
```

Abstract Method

An abstract method is a method that is declared without an **implementation**.



```
public abstract class Room {  
  
    public abstract double calculateTotalAmount(int numberOfDays);  
}
```

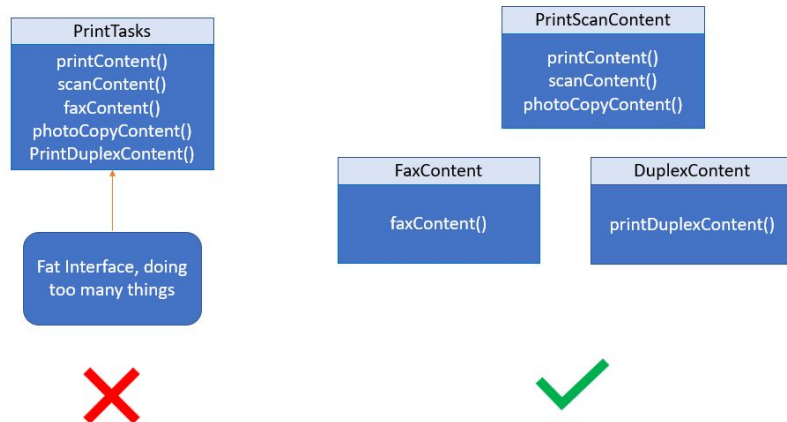
Interface

Is abstract type used to specify the behavior of a class. An interface in Java is a **blueprint** of a behavior.

```
public interface IMovable {  
    void move();  
}
```

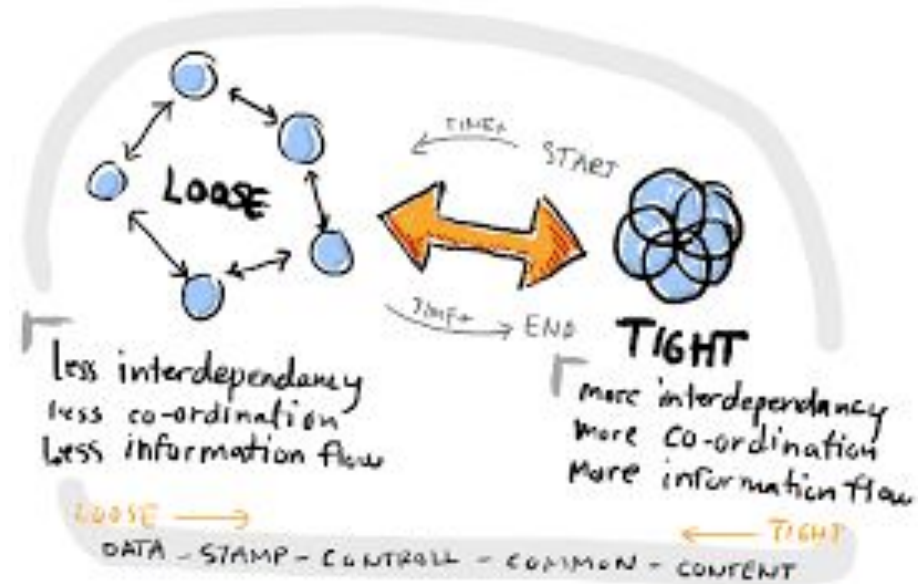
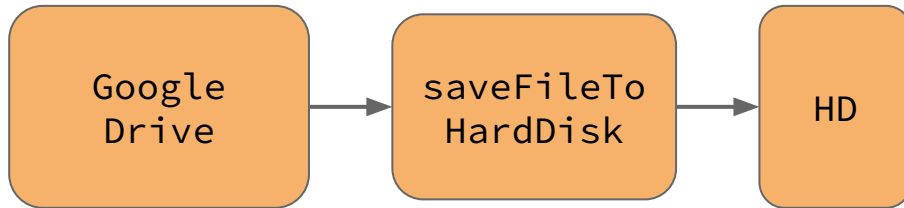
Interface Segregation

A principle states that clients should not be forced to depend upon interfaces that they **do not** use.



Coupling

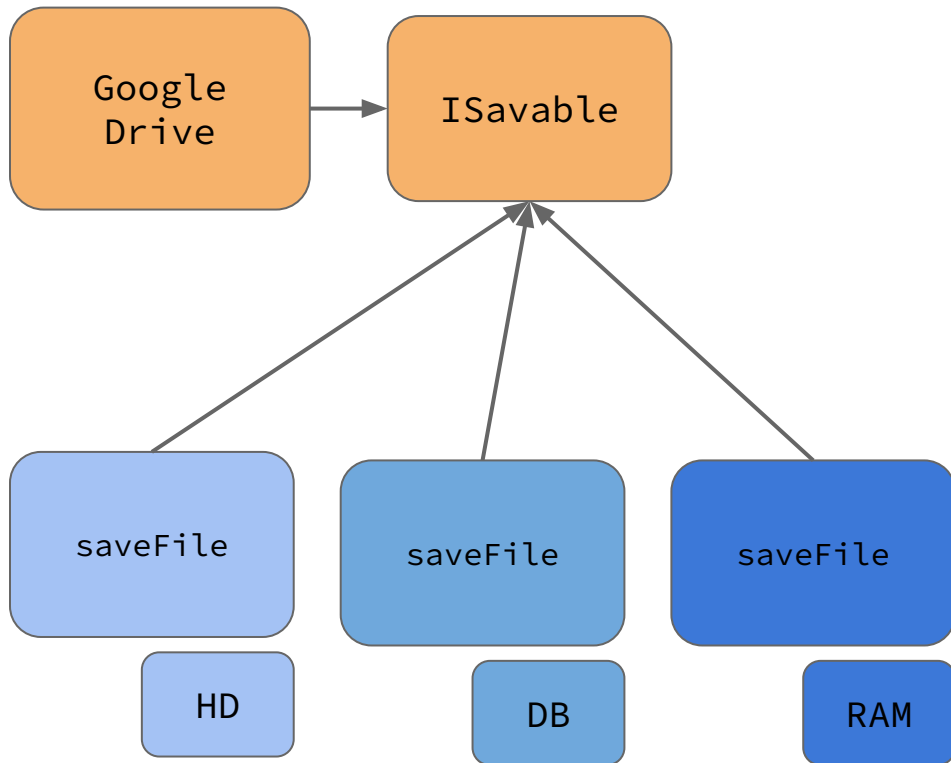
The degree of **dependency** between software components



Decoupling / Breaking Dependencies

In order to reduce coupling between classes you need to follow the rule that says, always Depend on **abstractions**, not on **concretions** which means, coding to interfaces, not to concrete classes as much as possible.

By this you now follow **Dependency Inversion Principle**



Interface VS Abstract Class

Besides the implementations differences between abstract class and interface, the main logically difference is usually we are using the abstract class to represent a specific **kind/entity**, on the other hand we usually use interface to represent a **behavioural** abstraction, or even just a **marker** of some classes to fire some logic based on that.

Polymorphism

Is the ability of an **component** to take on **many forms**, The word itself indicates the meaning as poly means many and morphism means types.

Built on top of Inheritance



arbabwaseer@gmail.com

Polymorphism Forms

Method
Override

Interchangeable
Objects

Method
Overload

Method Override

is a feature that allows a subclass or child class to provide a **specific implementation** of a method that is already provided by one of its super-classes or parent classes.

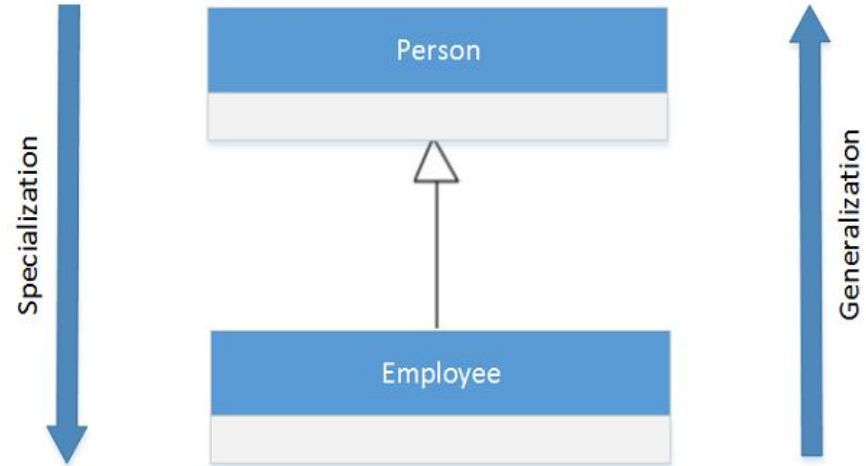
Overriding

```
class Dog{
    public void bark(){
        System.out.println("woof ");
    }
}
class Hound extends Dog{
    public void sniff(){
        System.out.println("sniff ");
    }
    public void bark(){
        System.out.println("bowl");
    }
}
```

Same Method Name,
Same parameter

Interchangeable Objects

Is the ability to **replace** an object with another object from the same hierarchy as result of using inheritance, by this we can get benefits like **code generalization** on top of that hierarchy.



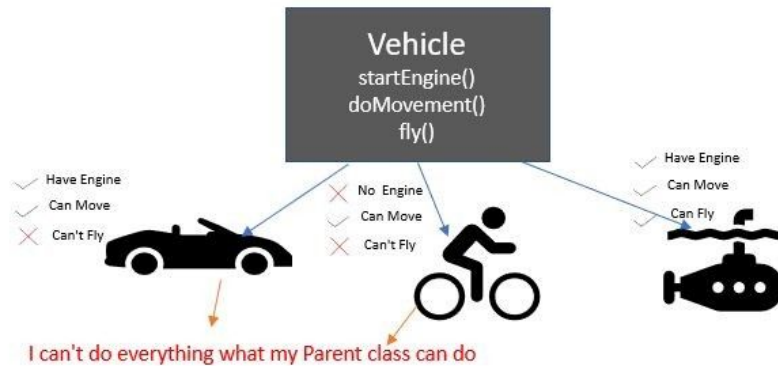
The Employee class extends the Person class (Inheritance)

Liskov Substitution Principle

Objects of a superclass should be **replaceable** with objects of its subclasses **without breaking** the application.

It's a principle that measures how **accurate** your inheritance relationship is.

If you found yourself adding conditional statements to handle specific logic for general use case for any class hierarchy or inheritance relationships, that means you are violating LSP and your inheritance isn't correct.



Conclusion

What Benefits we Got From Using OOP

- We are using constructors now in an efficient way.
- We are now adding more methods in classes, not just attributes and getters or setters.
- We are now have more flexibility over code that we can add or change any part easily.
- We are now having more logical structure of code, that represents each entity and its role.
- We are now having more organized code easy for debugging and fixing.

Structured VS Object Oriented

Structured Programming

- Simple way of implementation.
- Produce fragile software.
- Better choice with small software or a group of functionalities that don't need a huge interaction.

Object Oriented Programming

- A little bit complex way of implementation.
- Produce a flexible and maintainable application.
- Better choice be used with medium to large softwares that manage a complex domain.